

CI/CD パイプライン + クラウド

移行ソリューションのご提案

ProQuip 調達・在庫管理システム

オンプレミス手動運用 → GitHub Actions + OpenShift 自動化

2026年6月

本日のアジェンダ (30分)

時間	内容
5分	現状の課題 — オンプレミス手動運用のリスク
10分	提案ソリューション — CI/CD + クラウドの仕組み
5分	デモ — 実際のパイプライン動作
5分	コスト比較 — 3年間TCOと投資回収
5分	質疑応答

1. 現状の課題

オンプレミス手動運用のリスク

今のデプロイ、こうなっていないませんか？

- | | | |
|--------------------------|-------|----------|
| 1. 開発者がローカルで Maven ビルド | (30分) | |
| 2. EAR ファイルを手動でサーバーにコピー | (15分) | |
| 3. WildFly を停止 | (5分) | ← サービス停止 |
| 4. 旧バージョンのバックアップ | (15分) | |
| 5. 新バージョンをデプロイ | (10分) | |
| 6. WildFly を起動 | (10分) | ← サービス復旧 |
| 7. 手動でブラウザテスト | (30分) | |
| 8. フロントエンドのビルド・配置 | (20分) | |
| 9. nginx 設定確認 | (10分) | |
| 10. DB マイグレーション (手動 SQL) | (30分) | |
| 11. 動作確認・報告書作成 | (60分) | |

合計： 約 4～8 時間

手動運用の4大リスク

● 人的ミス

- デプロイの **15～25%** が失敗
- 「設定ファイルを間違えた」
- 「SQL の実行順を間違えた」

● 属人化

- 「田中さんがいないとデプロイできない」
- 手順書が古い / 存在しない
- パスワードを知っているのは2人だけ

● サービス停止

- デプロイ中は **必ず** サービス停止
- 深夜・休日に実施が必要
- ロールバックに **1～2時間**

● リリース遅延

- デプロイが怖い → **月1回** しかリリースできない
- バグ修正も次のリリースまで待つ
- ビジネスの俊敏性を阻害

障害事例: よくあるシナリオ

金曜 22:00 — デプロイ開始

23:30 — DB マイグレーション失敗。ロールバック開始

翌 01:00 — バックアップから復元完了。だが一部データが不整合

翌 03:00 — 手動でデータ修正。ようやく復旧

月曜 09:00 — 報告書作成、再発防止策の会議...

5時間のダウンタイム + 週末の深夜対応

これが年に **2~3回** 発生している

2. 提案ソリューション

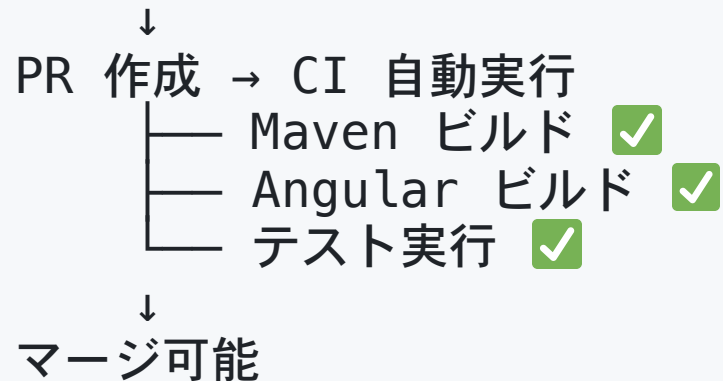
CI/CD + クラウドによる完全自動化

CI/CD とは？

CI — 継続的インテグレーション

コード変更のたびに **自動でビルド・テスト**

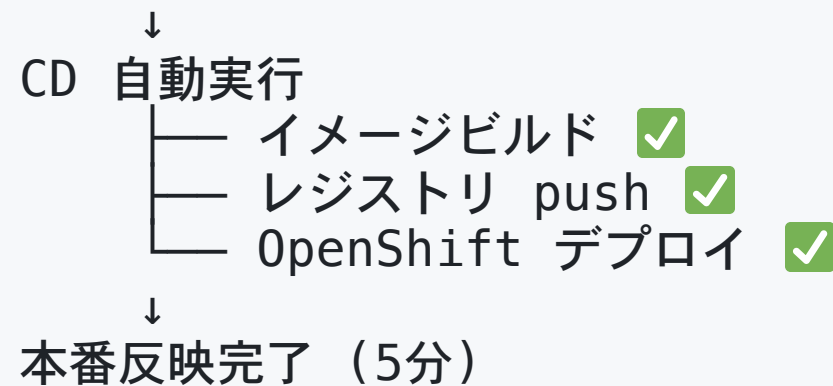
開発者がコード変更



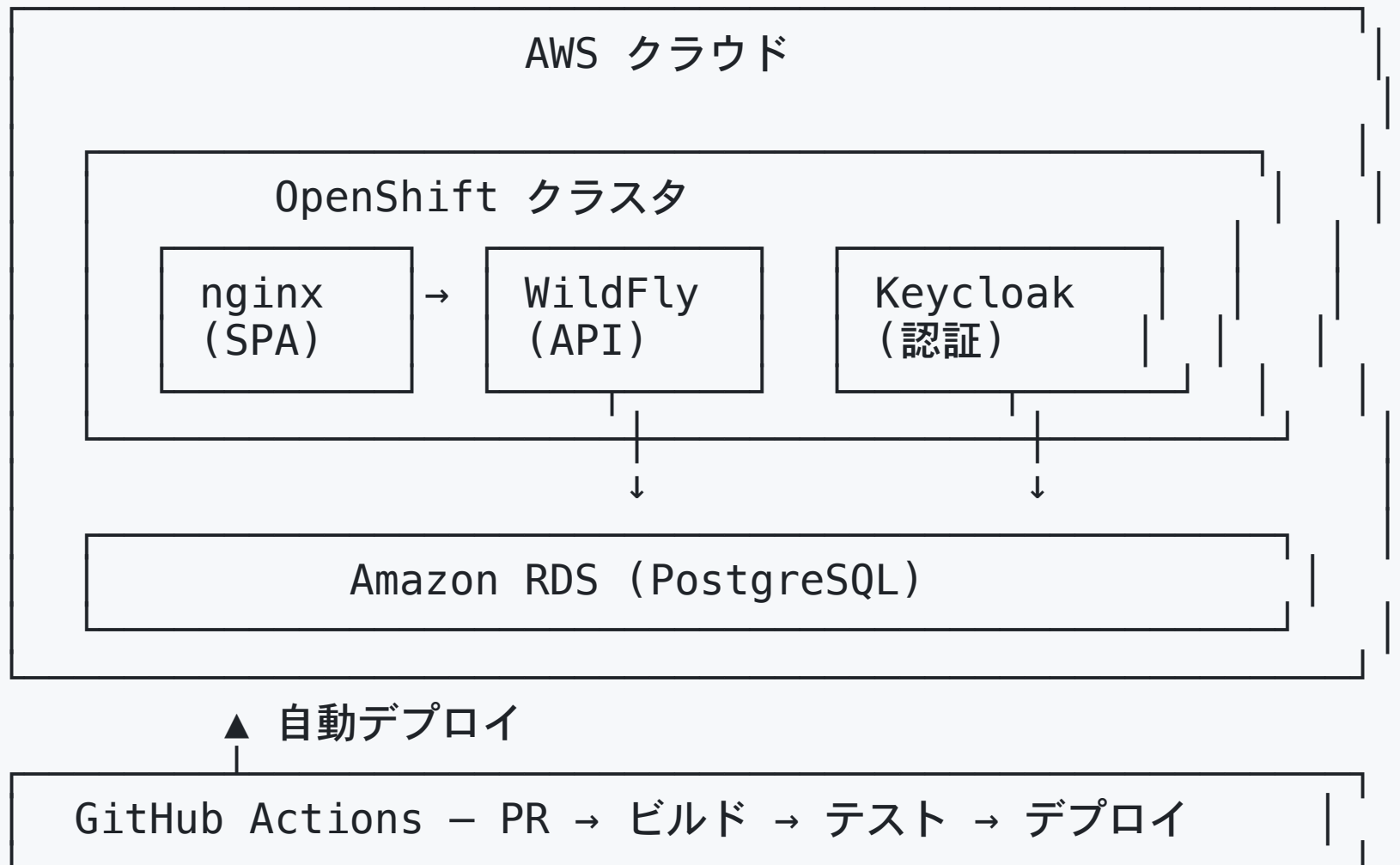
CD — 継続的デリバリー

マージ後に **自動でデプロイ**

main マージ



アーキテクチャ全体像



Before / After 比較

作業	従来 (手動)	CI/CD (自動)
コードビルド	ローカルで <code>mvn package</code>	GitHub Actions が自動実行
テスト実行	手動 or 省略	PR ごとに 自動実行
ビルド成果物の転送	SCP / FTP	コンテナイメージ自動 push
アプリ停止・起動	SSH でサーバー操作	ローリングアップデート (停止なし)
DB マイグレーション	手動 SQL	Flyway が 自動実行
動作確認	手動ブラウザテスト	自動ヘルスチェック
ロールバック	バックアップから手動復元	<code>oc rollout undo</code> (30秒)
設定管理	Excel / テキスト	ConfigMap / Secret (バージョン管理)

コンテナ化のメリット

従来: 「環境差異」問題

開発者PC → ビルド成功 
検証サーバー → 動作 OK 
本番サーバー → 動かない 

「なぜ本番だけ...？」

→ Java バージョン違い、OS パッチ差異、
設定ファイルの不整合...

CI/CD: 「同一イメージ」保証

GitHub Actions → イメージビルド
↓
同じイメージを全環境にデプロイ
├─ 検証環境 
└─ 本番環境 

「開発で動けば本番でも動く」
→ コンテナに全てパッケージ済み

ダウンタイムゼロのデプロイ

従来: 停止を伴うデプロイ

[旧バージョン 稼働中]
↓ 停止 **×** サービスダウン
[サーバー停止中...]
↓ 起動
[新バージョン 稼働中]

停止時間: 10~30分

CI/CD: ローリングアップデート

[旧 Pod ■■■ 稼働中]
[新 Pod □□□ 起動中...]
↓ 準備完了
[旧 Pod ■■ 稼働中]
[新 Pod ■□□ 稼働中]
↓ 段階的に切替
[新 Pod ■■■ 稼働中]

停止時間: 0秒

ロールバックの比較

従来: 1～2時間

1. 障害発生を **人が検知**
2. 担当者に **電話連絡**
3. サーバーに SSH
4. バックアップを **探す**
5. 旧バージョンに **差し替え**
6. DB を **手動で戻す**
7. 動作確認

 深夜に1～2時間の作業

CI/CD: 30秒

```
oc rollout undo deployment/proquip-wildfly
```

1. コマンド **1行** 実行
2. OpenShift が自動で旧バージョンに切替
3. ヘルスチェック自動実行

 **30秒で完了、誰でも実行可能**

3. デモ

実際のパイプライン動作

デモ: コード変更 → 自動デプロイ

実演フロー (約5分)

1. コードを変更 — 1行修正
2. PR を作成 — `gh pr create`
3. CI が自動起動 — Maven ビルド + Angular テスト
4. レビュー承認 → マージ
5. CD が自動起動 — イメージビルド → レジストリ push → デプロイ
6. ブラウザで確認 — 変更が本番に反映されている

所要時間: PR 作成からデプロイ完了まで約5分

従来の手動デプロイ: 4～8時間

デモ: Branch Protection

マージ前に自動で品質チェック

PR 作成

↓

- ✓ Backend Build & Test – Maven ビルド成功
- ✓ Frontend Build & Test – Angular ビルド成功
- ✓ 1 Approve – レビュー承認済み

↓

マージ可能 [Merge] ボタンが緑に

テストが失敗 or レビュー未承認 → マージ不可能

「うっかりバグを本番に入れる」事故を 技術的に防止

4. コスト比較

3年間TCOと投資回収

デプロイ時間の比較

従来

4～8h

1回のデプロイ所要時間

月1～2回 × 2～3名体制

CI/CD

5min

1回のデプロイ所要時間

98% 短縮

自動実行、人間の作業ゼロ

年間デプロイ作業コスト比較

項目	従来 (手動)	CI/CD (自動)
デプロイ作業 (インフラ担当)	96h × ¥5,000 = ¥480,000	¥0 (自動)
テスト・確認 (QA)	48h × ¥4,000 = ¥192,000	¥0 (自動)
立会い (開発リーダー)	72h × ¥6,000 = ¥432,000	¥0 (不要)
障害対応 (年6回)	24h × ¥7,000 = ¥168,000	1h × ¥7,000 = ¥7,000
休日・深夜手当	¥360,000	¥0
PR レビュー・パイプライン保守	—	¥192,000
年間合計	¥1,728,000	¥199,000
		88% 削減

3年間 TCO 比較

コスト区分	オンプレミス (3年)	CI/CD + クラウド (3年)
ハードウェア / インフラ	¥10,860,000	¥3,744,000
ソフトウェアライセンス	¥3,000,000	¥0
人件費 (運用 + デプロイ)	¥15,984,000	¥4,917,000
合計	¥28,644,000	¥8,861,000

3年間で ¥19,783,000
削減 (69%)

投資回収

指標	値
初期投資 (パイプライン構築)	約 ¥200,000
年間コスト削減額	約 ¥6,600,000
投資回収期間	約 11日
3年間 ROI	9,792%

構築実績: ゼロから本番稼働まで **2時間23分** で完了

(AWS 基盤準備 25分 + CI/CD 構築 31分 + コンテナ対応 15分 + デプロイ 69分 + 検証 4分)

DORA メトリクス比較

指標	従来	CI/CD 導入後	業界 Elite
デプロイ頻度	月1～2回	日1～5回	日複数回
リードタイム	2～4週間	数時間	15分未満
デプロイ失敗率	15～25%	5%未満	0.5%未満
障害復旧時間	2～4時間	15分	7分未満

DORA (DevOps Research and Assessment) は Google が支援するソフトウェアデリバリーパフォーマンスの研究プロジェクト

リスク比較

リスク	従来	CI/CD
デプロイ時の人的ミス	● 高	● 極低
属人化	● 高	● 低
環境差異	● 高	● なし
ロールバック失敗	● 中	● 極低
セキュリティパッチ遅延	● 高	● 即座
ハードウェア障害	● 高	● 自動復旧
スケーラビリティ	● 低	● 高
監査証跡	● 手動	● 自動

移行ロードマップ

週	作業内容	担当
1週目	AWS/OpenShift 環境準備、CI/CD パイプライン構築	インフラ + DevOps
2週目	Dockerfile 修正、コンテナイメージ構築	開発チーム
3週目	ステージング環境でのテスト	QA チーム
4週目	本番切り替え + 並行運用	全員

提供するもの

-  11本のステップバイステップ チュートリアル
-  CI/CD ワークフロー (ci.yml + cd.yml)
-  OpenShift マニフェスト一式 (k8s/)
-  ビルド・デプロイスクリプト

まとめ

3つのポイント

- 1 デプロイ時間: 4～8時間 → 5分 (98% 短縮)
- 2 3年間コスト: ¥2,864万 → ¥886万 (69% 削減)
- 3 人的ミス: 年6回以上の障害 → ゼロに近い自動化

次のステップ

1. **PoC (概念実証)** — 1週間で検証環境にパイプライン構築
2. **チーム教育** — チュートリアルを使った実践ワークショップ (1日)
3. **段階的移行** — ステージング → 本番の順で切り替え

お問い合わせ

チュートリアル・ドキュメント一式は
GitHub リポジトリ でご提供いたします

ご質問・ご相談

Q&A

付録: 技術スタック

コンポーネント	技術	役割
ソースコード管理	GitHub	バージョン管理、PR、コードレビュー
CI パイプライン	GitHub Actions	自動ビルド、自動テスト
CD パイプライン	GitHub Actions + oc CLI	自動デプロイ
コンテナレジストリ	OpenShift 内部レジストリ	イメージ保管
コンテナ基盤	OpenShift (AWS)	アプリケーション実行環境
アプリケーションサーバー	WildFly 30 (Jakarta EE 10)	バックエンド API
フロントエンド	Angular 17 + nginx	SPA 配信
認証	Keycloak 22 (OIDC)	シングルサインオン
	Amazon RDS (PostgreSQL)	

付録: チュートリアル一覧

#	チュートリアル	対象者
1.1	CLI ツールのインストール	全員
1.2	OpenShift クラスタへの接続	インフラ
1.3	コンテナレジストリの設定	インフラ
1.4	RDS PostgreSQL の作成	インフラ
2.1	GitHub Secrets の設定	DevOps
2.2	CI ワークフローの作成	開発
2.4	CD ワークフローの作成	DevOps
2.6	Branch Protection の設定	リーダー
3.x	コンテナイメージの OpenShift 対応	開発
4-5	OpenShift デプロイと検証	全員